COURSE-BASED PROJECT DOCUMENTATION | DOC-11

Native C Backend (DLL & EXE)

C ABI Export, Memory Buffers, Return Codes, & Compilation

1. C Backend Architecture

The parsing core is implemented in C99, linking the Bison-generated parser (java_parser.tab.c) and Flex-generated scanner (lex.yy.c). It is compiled with GCC into both a shared dynamic link library (javacheck.dll) and a standalone command-line executable (javacheck.exe).

2. C ABI Export: check_syntax()

```
#ifdef _WIN32
__declspec(dllexport)
#endif
int check_syntax(const char *filepath, char *err_msg, int err_msg_len) {
    // 1. Reset state variables (yylineno, current_column, error_line, yylloc)
    // 2. Open file stream: fopen(filepath, "r")
    // 3. Re-initialize scanner: yyrestart(f); yyin = f;
    // 4. Execute parser: int result = yyparse();
    // 5. Close file and write formatted message to err_msg
    // 6. Return 0 (Success), 1 (Syntax Error), or 2 (File Error)
}
```

3. Standard Return Code Contract

- ? Return 0: Valid Java syntax; err_msg = 'SUCCESS: Valid Java Syntax'
- ? Return 1: Syntax error detected; err_msg = 'ERROR: Line X: Syntax error near 'token''
- ? Return 2: System or file I/O error; err_msg = 'ERROR: Could not open file '...'

4. Native Compilation Commands

```
# Compile Shared Library (DLL)
gcc -O2 -shared -Iparser -Ilexer parser/java_parser.tab.c lexer/lex.yy.c -o javacheck
  .dll

# Compile Standalone CLI Executable
gcc -O2 -Iparser -Ilexer parser/java_parser.tab.c lexer/lex.yy.c -o javacheck.exe
```